

DevOps & Cloud Interview — Round 1

Complete Q&A Reference | Core Fundamentals

■ Docker

Q: Define a Dockerfile and its role.

- A Dockerfile is a text file containing sequential instructions to build a Docker image.
- Each instruction (FROM, RUN, COPY, CMD, EXPOSE, ENV...) creates a read-only image layer.
- Role: codifies environment setup — runtime, dependencies, config — into a repeatable, version-controlled artifact.
- Key instructions:
 - FROM — base image (always first)
 - RUN — executes commands during build (installs packages)
 - COPY / ADD — moves files from host into the image
 - CMD / ENTRYPOINT — defines the default container process
 - EXPOSE — documents the port the app listens on
 - ENV — sets environment variables

■ *Analogy: Dockerfile = recipe; Docker image = baked dish; Container = served plate.*

Q: How do you secure a Docker image?

- Use a minimal base image (distroless, alpine) to shrink the attack surface.
- Never run as root — add a non-root USER in the Dockerfile.
- Multi-stage builds: compile in one stage, copy only the binary to a clean final stage.
- Pin image versions (FROM node:20.11-alpine, not FROM node:latest).
- Scan images with Trivy or Snyk before pushing:
 - trivy image myapp:latest
- Remove secrets from layers — use build-time ARGs or runtime env vars, never COPY of .env files.
- Sign images with Docker Content Trust (DCT) / Cosign (Sigstore).
- Set a read-only filesystem at runtime: docker run --read-only.
- Apply resource limits: --memory and --cpus flags.

Q: Scenario: How do you securely store and distribute Docker images?

- Storage:
 - Use AWS ECR (private registry) — images stored encrypted at rest with KMS.
 - Enable ECR image scanning on push (uses Clair/Trivy under the hood).
 - Apply ECR lifecycle policies to expire old/untagged images.
- Access control:
 - IAM roles/policies control who can push/pull — never share long-term credentials.
 - Use IRSA (IAM Roles for Service Accounts) in EKS so pods pull images without static secrets.
- Distribution:
 - Tag images with Git SHA + semantic version (myapp:1.4.2-abc1234), not just 'latest'.
 - Use immutable ECR tags to prevent overwriting production images.
 - Replicate ECR across regions for low-latency pulls in multi-region deployments.
- CI/CD integration:
 - GitHub Actions authenticates to ECR via OIDC — no long-lived AWS keys in CI secrets.

■ Infrastructure as Code (IaC)

Q: CloudFormation vs Terraform — when to use each?

- CloudFormation — choose when:
 - You are 100% AWS-only and want native integration (StackSets, Service Catalog, CDK output).
 - You need drift detection built into the console with no extra tooling.
 - AWS releases a new service — CFN templates often land same day.
 - Terraform — choose when:
 - Multi-cloud or hybrid: AWS + GCP + Azure, or AWS + Datadog + GitHub providers in one codebase.
 - Team already knows HCL and benefits from a rich provider ecosystem (1,000+ providers).
 - You need remote state, workspaces, and a mature module registry.
 - Key differences:
 - State: Terraform manages state explicitly (S3 + DynamoDB lock); CFN manages stack state internally.
 - Rollback: CFN rolls back the entire stack on failure automatically; Terraform requires targeted destroy/apply.
 - Language: CFN = JSON/YAML; Terraform = HCL (more readable, supports loops/conditionals natively).
- At Hisan Labs I used Terraform for AWS infra — it gave us reusable modules across dev/staging/prod.

Q: What is Terraform State?

- State is Terraform's source of truth — a JSON file mapping your HCL config to real infrastructure resource IDs.
- Without state, Terraform cannot know what already exists; every plan would try to recreate everything.
- Remote state (best practice):
 - Store in an S3 bucket with versioning enabled (audit trail of every apply).
 - Use DynamoDB table for state locking — prevents two engineers running terraform apply simultaneously.
 - Example backend config:
 - backend "s3" { bucket="tf-state-prod", key="eks/terraform.tfstate", region="ap-south-1", dynamodb_table="tf-lock" }
- Sensitive data warning: state files can contain secrets — encrypt the S3 bucket with KMS and restrict access via IAM.

Q: What happens with configuration drift?

- Drift = when real infrastructure diverges from the state Terraform or CFN believes it to be in.
- Causes: manual changes in AWS Console, auto-scaling events, other teams modifying shared resources.
- Detection:
 - Terraform: terraform plan shows drift as unexpected diffs (~ update or - destroy).
 - CloudFormation: Drift Detection marks resources as MODIFIED or DELETED.
- Remediation options:
 - Import drift: terraform import (pull the real resource back into state).
 - Overwrite drift: terraform apply forces resources back to declared config.
 - Accept drift: update the IaC code to codify the intentional change.
- Prevention:
 - Block console access for infra resources via SCPs (Service Control Policies).
 - Run terraform plan in CI on a schedule to catch drift early.

■ Monitoring & Observability

Q: Prometheus vs Grafana — what is the purpose of each?

- Prometheus:
 - Time-series database and metrics collection engine.
 - Scrapes /metrics endpoints from targets (apps, exporters, Kubernetes nodes).
 - Stores metrics locally; provides PromQL for querying.
 - Includes Alertmanager for firing alerts to Slack, PagerDuty, email.
 - Grafana:
 - Visualization and dashboarding layer — it does NOT store data.
 - Connects to Prometheus (and 50+ other data sources) via data source plugins.
 - Lets you build dashboards, set visual thresholds, and create alerts with a UI.
 - How they work together:
 - Prometheus collects and stores → Grafana queries Prometheus and renders dashboards.
 - Rule of thumb: Prometheus = backend brain; Grafana = frontend eyes.
- At Hisan Labs: Prometheus scraped EKS pod metrics; Grafana showed CPU/memory dashboards per namespace.
-

Q: Metric vs Trace — what is the difference?

- Metric:
 - A numeric measurement aggregated over time. Examples: CPU %, request rate (req/s), error count.
 - Answers: 'How much?' and 'How fast?' — great for alerting and dashboards.
 - Low storage cost; easy to graph trends.
 - Trace:
 - A record of a single request's journey across multiple services/components.
 - Made up of spans — each span = one unit of work (DB query, API call, function).
 - Answers: 'Why is this request slow?' — pinpoints the exact bottleneck.
 - Tools: Jaeger, Zipkin, AWS X-Ray, OpenTelemetry.
 - Analogy:
 - Metric = speedometer of your car (aggregate number).
 - Trace = GPS journey log of a single trip (detailed path).
 - Third pillar — Logs: timestamped text events; Metrics + Traces + Logs = full observability.
-

■ AWS

Q: CloudFront use cases — when and why do you use it?

- CloudFront is AWS's CDN (Content Delivery Network) with 450+ global edge locations.
 - Common use cases:
 - Static website / SPA delivery: cache S3-hosted React/Vue assets at edge — sub-50ms loads globally.
 - API acceleration: cache GET responses from ALB/API Gateway; reduce origin load.
 - Video streaming: HLS/DASH video via CloudFront + MediaPackage — adaptive bitrate, global scale.
 - DDoS protection: CloudFront + AWS Shield Standard absorbs volumetric attacks at the edge.
 - Security layer: attach WAF (Web Application Firewall) rules to block SQLi, XSS, bad bots.
 - Signed URLs/Cookies: restrict access to premium content to authenticated users only.
 - Key benefit: reduces latency by serving cached content from the nearest PoP instead of hitting your origin every time.
-

Q: Subnets and Security Groups — why do they matter?

- Subnets (network-level segmentation):
 - Public subnet: has a route to the Internet Gateway — for load balancers, bastion hosts.
 - Private subnet: no direct internet route — for EKS worker nodes, RDS, app servers.
 - Database subnet: further isolated private subnet — only accepts traffic from app subnet.
 - Why: limits blast radius if a component is compromised; public internet can't reach your DB directly.
 - Security Groups (instance-level stateful firewall):
 - Whitelist-only model — default deny all; you add explicit allow rules.
 - Stateful: if you allow inbound on port 80, the response is automatically allowed outbound.
 - Best practice: reference SG IDs in rules instead of CIDR blocks — e.g. allow SG-ALB on port 8080 instead of 0.0.0.0/0.
 - Together: Subnets = which room in the house; Security Groups = lock on the door of that room.
-

Q: Cost optimization strategies you have used.

- Right-sizing: analyze CloudWatch CPU/memory metrics, downsize over-provisioned EC2 instances.
 - Reserved Instances / Savings Plans: commit to 1-3 year plans for baseline workloads — up to 72% savings.
 - Spot Instances: use for stateless, fault-tolerant workloads (EKS batch jobs, CI runners) — up to 90% cheaper.
 - S3 lifecycle policies: auto-transition objects to S3-IA (Infrequent Access) after 30 days, Glacier after 90 days.
 - NAT Gateway optimization: consolidate traffic through fewer NAT Gateways; avoid cross-AZ data transfer charges.
 - ECR lifecycle policies: auto-delete untagged images older than 14 days — reduces storage cost.
 - CloudWatch log retention: set log group retention (e.g. 30 days) instead of keeping logs indefinitely.
 - AWS Cost Explorer + Budgets: set monthly budget alerts; use Cost Explorer to identify top spend categories.
- *Practical tip: data transfer costs are often the hidden budget killer — always check cross-AZ and internet egress.*
-

■ Serverless

Q: Lambda vs EC2 — when do you choose which?

- Choose Lambda when:
 - Event-driven, short-duration tasks (S3 trigger, API Gateway request, SQS consumer).
 - Unpredictable or spiky traffic — Lambda scales to zero and to thousands of concurrent executions automatically.
 - You want zero server management — no patching, no AMI updates, no capacity planning.
 - Max execution time is under 15 minutes.
 - Choose EC2 when:
 - Long-running processes (> 15 min): ML training, video transcoding, batch ETL jobs.
 - Stateful workloads that need persistent local disk or in-memory state.
 - Consistent, predictable high load — EC2 + Reserved Instances is cheaper than Lambda at sustained scale.
 - Legacy apps that require specific OS configs, custom kernel modules, or GPU access.
 - Decision matrix:
 - Spiky + short-lived = Lambda | Steady + long-running = EC2 | Middle ground = ECS Fargate.
- *Hisana Labs project: 2048 Game on EKS Fargate — Fargate won because it needed persistent K8s pods but no EC2 management.*
-

■ Networking & Kubernetes

Q: SLI, SLO, SLA — definitions.

- SLI (Service Level Indicator):
 - The actual measured metric that reflects service health.
 - Examples: availability = (successful requests / total requests) × 100; latency = p99 response time.
 - SLO (Service Level Objective):
 - The internal target you set for an SLI — your reliability goal.
 - Example: 'Availability SLO = 99.9% per month' (allows ~43 min downtime/month).
 - SLOs drive engineering priorities — error budget = 100% - SLO.
 - SLA (Service Level Agreement):
 - The external contract with customers, with financial consequences for breach.
 - Example: AWS EC2 SLA = 99.99% — if breached, customers get service credits.
 - Relationship: SLI is measured → compared against SLO (internal goal) → SLA (external promise).
 - $SLA \leq SLO$ — you never promise customers more than your internal target.
-

Q: Kubernetes high-level architecture.

- Control Plane (manages the cluster):
 - kube-apiserver: the front door — all kubectl commands hit the API server.
 - etcd: distributed key-value store — the source of truth for all cluster state.
 - kube-scheduler: watches for unscheduled pods and assigns them to nodes based on resources/constraints.
 - kube-controller-manager: runs controllers (ReplicaSet, Deployment, Node) to reconcile desired vs actual state.
 - cloud-controller-manager: integrates with cloud provider APIs (AWS ELB, EBS).
 - Worker Nodes (run workloads):
 - kubelet: agent on each node — ensures containers in pods are running as specified.
 - kube-proxy: manages iptables/IPVS rules for Service networking and load balancing.
 - Container runtime: containerd or CRI-O — actually runs the containers.
 - Core objects: Pod → ReplicaSet → Deployment → Service → Ingress → ConfigMap / Secret.
- In EKS: control plane is fully managed by AWS; you manage worker nodes (managed node groups or Fargate).
-

Q: VPC Peering and interconnectivity.

- VPC Peering: a direct, private network connection between two VPCs (same or different accounts/regions).
 - Traffic travels over AWS backbone — never the public internet.
 - Key constraints:
 - No transitive peering: if A peers B and B peers C, A cannot reach C via B — you need A-C peering directly.
 - CIDR blocks must not overlap between peered VPCs.
 - You must update route tables in both VPCs and add SG rules to allow traffic.
 - Alternatives:
 - AWS Transit Gateway: hub-and-spoke model — connect 100s of VPCs and on-prem networks via one gateway. Supports transitive routing.
 - AWS PrivateLink: expose a specific service from one VPC to another without full VPC peering (more secure, no CIDR overlap requirement).
-

Q: VPC to On-premises connectivity — which gateways?

- Option 1 — AWS Site-to-Site VPN:
 - Encrypted IPSec tunnel over the public internet between your VPC and on-prem router.
 - Components: Virtual Private Gateway (VGW) on the AWS side + Customer Gateway (CGW) on the on-prem side.
 - Throughput: up to 1.25 Gbps per tunnel; simple to set up; suitable for smaller workloads.
 - Option 2 — AWS Direct Connect (DX):
 - Dedicated private fiber link between your data center and an AWS Direct Connect location.
 - No internet traversal — consistent latency, higher bandwidth (1/10/100 Gbps).
 - More expensive and takes weeks to provision; best for large data transfers and compliance requirements.
 - Option 3 — Direct Connect + VPN (hybrid):
 - Direct Connect as primary path; VPN as encrypted failover — best of both worlds for production.
 - Transit Gateway also acts as a VPN termination point when connecting multiple VPCs to on-prem simultaneously.
- *Interview tip: Site-to-Site VPN = quick and cheap; Direct Connect = enterprise-grade, low-latency, no internet.*